PROJECT REPORT | MILESTONE 1

BookNest

A reading and writing academy with a small online bookshop.

TEAM	MEMBER	STUDENT ID	UNIVERSITY EMAIL
Hossam w Ibrahim	Youssef Hisham	16003630	Youssef.hisham@student.giu-uni.de
	Omar Ogaiz	16002864	Omar.ogaiz@student.giu-uni.de

PROJECT IDEA

BookNest brings a creative academy and an online bookshop into one student-facing experience. Students can discover reading and writing workshops, enroll in courses, browse books and stationery, and keep their wishlist, orders, and fee history together.

The prototype also gives staff simple tools to track course invoices, payroll, department income and expenses, and bookshop stock. It is designed as a front-end coursework milestone using local sample data rather than a live financial service.

WHO USES IT

STUDENT	ACCOUNTANT	ADMIN
Create a profile, join workshops, shop, save books, check out, and review fees and order history.	Review student fee records, mark invoices paid, edit payroll, and compare department budgets.	Manage products and stock; view people, access, budgets, and payroll records.

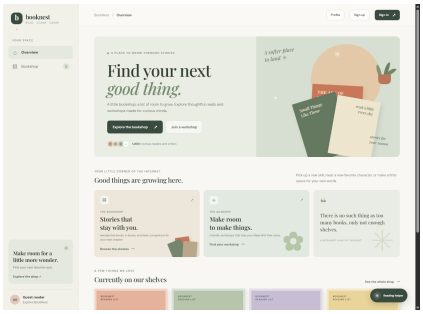
01 / STUDENT EXPERIENCE

A calm place to learn and shop

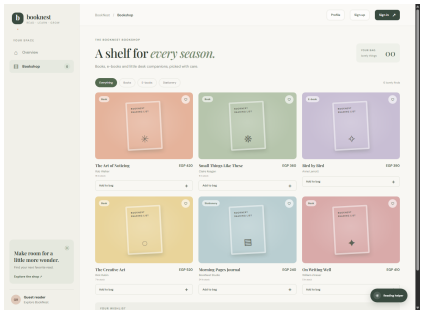
The home page introduces the academy and shop and links to sign-in and sign-up. Lazy-loaded pages keep the main entry experience small while the rest of the app loads when opened.

LEARN	SHOP	KEEP TRACK
Browse three writing and reading workshops; enroll and see course fees.	Filter the catalog, wishlist items, use a session cart, check out, and view order history.	Review invoices, mark a fee paid in the demo, and download fee history as JSON.

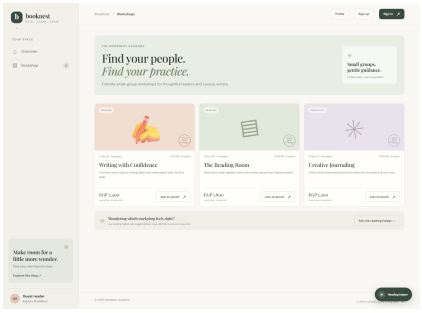
SAMPLE RUNS | SCREENSHOTS



Home: academy and shop entry



Bookshop: browse and shop



Workshops: course enrollment

SAMPLE RUN

A student enrolls in a workshop, shops for a book, checks out, and reviews the order in their profile. An accountant creates an invoice, records payment, and sees the updated status in the fee table.

MANAGEMENT VIEWS

Accountants create student invoices, record payment status, edit staff pay, allowances and deductions, and export monthly payroll JSON. The finance page uses a Chart.js bar chart to compare income and expenses for Bookshop and Workshops; accountants edit department figures and export the monthly report as JSON. Admins manage products and stock, and view people, access, budgets, and payroll records.

Responsive layouts support narrow screens with compact navigation and horizontally scrollable data tables. The reading helper calls Azure AI Foundry when local settings are present and otherwise falls back to course-based sample suggestions.

02 / IMPLEMENTATION & DATA

Simple tools, clear boundaries

The app uses browser storage because Milestone 1 is a front-end prototype. Each choice matches the lifetime of the data and is easy to inspect in a browser during a lab demonstration.

STORAGE	USED FOR	REASON
Cookie	A first-visit marker only.	Demonstrates a small browser cookie without placing profile or payment information in it.
Session storage	Signed-in demo session and shopping bag.	Short-lived data clears with the browser session; the cart need not remain after the session ends.
Local storage	Demo account/profile, preferences, wishlist, enrollments, orders, fees, payroll, department figures, products and stock.	Keeps sample data on the same device across visits so students and staff can review their previous demo actions.

TIME SAVING

Centralizing enrollment, invoices, orders, payroll and department totals in one set of screens reduces repeated lookups. JSON exports give staff a quick monthly record without retyping totals.

PRIVACY AND SECURITY

Role-based routes and navigation provide student, accountant, and admin experiences. Because this prototype stores data in the browser, it is suitable for sample information rather than confidential records; browser data can be inspected or changed on the device. Demo passwords are stored locally. The card number is a fixed all-zero mock value, only its last four digits appear in the profile, and no payment is processed. When enabled, the AI chat sends the question, course enrollment names, and sample catalog to Azure AI Foundry.

IMPLEMENTATION AND CHECKS

The project uses Angular standalone components, reactive forms with validation, lazy-loaded routes, simple signals for shared front-end state, and Chart.js. The code stays at the course level: there is no application backend, database, external authentication, or real payment integration.

Verification: the production build completes successfully. The Jasmine/Karma suite passed all 9 tests, covering totals, wishlist and cart behavior, checkout and stock updates, enrollment, product creation, demo signup/sign-in, payroll, department edits, shared fee status, invoice creation, and the app shell. The configured live AI chat was also exercised successfully.

PROTOTYPE SCOPE

Balances, products, and payroll are sample data stored in the browser. The application has no server-side database, external authentication, or payment processing. The AI chat uses Azure AI Foundry when configured and provides local catalog-based suggestions when the service is unavailable.